

Universidad Nacional de Río Cuarto
Facultad de Ciencias Exactas, Físico-Químicas y Naturales
Departamento de Computación
Licenciatura en Ciencias de la Computación

Bases de Datos II

Resumen Teórico

Práctico N.º 1 — Repaso de SQL

Alumno: Tomás Rodeghiero
Año: 2026

Índice

1. Introducción	2
2. SQL: visión general	2
3. DML: manipulación de datos	2
3.1. La sentencia SELECT	2
3.2. Cláusula WHERE : filtros sobre filas	3
3.3. Cláusula ORDER BY : ordenamiento	4
3.4. Funciones de agregación	4
3.5. GROUP BY : consultas sumarias	4
3.6. HAVING : filtros sobre grupos	5
3.7. Subconsultas	5
3.8. Reuniones (JOIN)	5
3.9. Operaciones sobre conjuntos	6
3.10. INSERT	6
3.11. UPDATE	6
3.12. DELETE	7
4. DDL: definición del esquema	7
4.1. Tipos de datos	7
4.2. CREATE TABLE	8
4.3. Ligaduras de integridad	8
4.4. Dominios (CREATE DOMAIN)	9
4.5. ALTER TABLE	10
4.6. Índices	10
4.7. Vistas (CREATE VIEW)	10
4.8. Borrado de estructuras: DROP	11
5. Conexión con los ejercicios del práctico	11
6. Diferencias entre motores: lo que no hay que olvidar	13
7. Ideas clave para repasar antes del parcial	13
Resumen final	14

1. Introducción

Bases de Datos II retoma los conceptos relacionales de Bases de Datos I y los profundiza hacia la administración real de motores: optimización, transacciones, concurrencia, programación en el servidor (procedimientos almacenados, *triggers*), XML/JSON, bases NoSQL, bases geográficas e inteligencia de negocios. El primer práctico es un **repaso integral de SQL** que se apoya en los dos primeros teóricos de la materia y deja preparada la base sobre la cual se construirán los temas posteriores.

El práctico tiene tres focos claros: definir esquemas con DDL respetando las restricciones del enunciado, poblarlos con datos coherentes mediante DML y escribir consultas combinando filtros, *joins*, agrupamiento y agregación. Como detalle no menor, el práctico se resuelve sobre los tres motores que la cátedra exige manejar: **MySQL 8+**, **PostgreSQL 14+** y **Oracle 10/11 XE**.

Idea clave. Aunque el SQL estándar es uno solo, cada motor tiene diferencias prácticas importantes (autoincrement, tipos numéricos, dominios, manejo de esquemas, etc.). Aprender a leer SQL en el estándar y traducirlo a cada motor es uno de los objetivos centrales del práctico.

2. SQL: visión general

SQL (*Structured Query Language*) es el lenguaje estándar para organizar, gestionar y recuperar datos en bases de datos relacionales. Históricamente nació como *SEQUEL* en un prototipo de IBM y fue luego estandarizado por ANSI e ISO. Con el tiempo el estándar incorporó *triggers* (SQL:1999), interacción con XML (SQL:2005), funciones polimórficas, JSON y funciones trigonométricas (SQL:2016), entre otras extensiones. En paralelo, los motores comerciales (Oracle, SQL Server, PostgreSQL, MySQL) ofrecen sus propias variantes y agregan funcionalidades fuera del estándar.

SQL se divide tradicionalmente en tres sublenguajes:

DML (Data Manipulation Language)

Manipulación de datos: consultas (**SELECT**) basadas en álgebra relacional, e inserción, modificación y borrado de filas (**INSERT**, **UPDATE**, **DELETE**).

DDL (Data Definition Language)

Definición de la estructura: creación, modificación y borrado de tablas, dominios, índices, vistas y demás objetos del esquema (**CREATE**, **ALTER**, **DROP**).

DCL (Data Control Language)

Control de acceso y permisos sobre los objetos (**GRANT**, **REVOKE**). Es el foco del Práctico 2, pero conviene reconocerlo desde ahora para ubicar cada sentencia en su categoría.

El Práctico 1 trabaja casi exclusivamente con **DDL** (Ejercicio 1) y **DML** (Ejercicios 2 y 3).

3. DML: manipulación de datos

3.1. La sentencia SELECT

SELECT es el comando más usado de SQL. Recupera datos de una o varias tablas y se organiza en seis cláusulas, dos obligatorias y cuatro opcionales:

```
SELECT [ALL|DISTINCT] { * | expr_columna_1 [AS alias_1] [, ...] }  
FROM nombre_tabla_1 [alias_1] [, nombre_tabla_n [alias_n]]
```

```
[WHERE condicionWhere]
[GROUP BY expr_columna_group1 [, expr_columna_group2] [...]]
[HAVING condicionHaving]
[{UNION [ALL] | INTERSECT | EXCEPT} SELECT ...]
[ORDER BY nombre_campo_1 [ASC|DESC] [, nombre_campo_2 [ASC|DESC]]];
```

Conceptualmente, el orden *lógico* en que el motor evalúa una consulta es distinto al orden en que se escribe: primero FROM (con sus JOINS), luego WHERE, después GROUP BY, luego HAVING, después la lista del SELECT y finalmente ORDER BY. Tener este modelo mental en la cabeza explica, por ejemplo, por qué un alias definido en el SELECT no siempre puede usarse en el WHERE.

DISTINCT y ALL. DISTINCT elimina filas repetidas del resultado. ALL (valor por defecto) las conserva. La diferencia es relevante cuando proyectamos sólo unas pocas columnas y muchas combinaciones se repiten.

Alias. Tanto las tablas como las columnas pueden recibir alias con AS. Es habitual abreviar las tablas (cliente c, factura f) para escribir condiciones de *join* más cortas.

3.2. Cláusula WHERE: filtros sobre filas

WHERE se usa para quedarse sólo con las filas que cumplan una condición lógica. Una condición es una *expresión*: compara columnas contra valores o contra otras columnas y combina sub-expresiones con operadores lógicos.

Operadores lógicos. AND, OR, NOT y XOR. Como en cualquier lenguaje, AND tiene mayor precedencia que OR, así que conviene siempre parentizar para evitar ambigüedades.

Operadores de comparación.

- Comparadores: =, <>, <, >, <=, >=.
- BETWEEN a AND b: pertenece al intervalo cerrado $[a, b]$; aplica a tipos numéricos, fechas y cadenas.
- LIKE 'patrón': comparación por patrón. % representa cualquier secuencia de caracteres y _ un único carácter.
- IN (subconsulta o lista): verdadero si el valor está dentro del conjunto. Muy usado para conectar consultas anidadas.
- EXISTS (subconsulta): verdadero si la subconsulta devuelve al menos una fila. Particularmente útil para subconsultas correlacionadas.
- IS NULL / IS NOT NULL: el único modo correcto de comparar contra NULL, ya que = NULL *nunca* es verdadero (la comparación da UNKNOWN).

Ejemplos sobre el esquema del teórico.

```
-- Artículos cuya descripción empieza con A.
SELECT * FROM articulos WHERE descr LIKE 'A%';

-- Empieza con A y termina con K.
SELECT * FROM articulos WHERE descr LIKE 'A%' AND descr LIKE '%K';

-- Artículos provistos por algún proveedor (subconsulta con IN).
SELECT * FROM articulos
WHERE Nart IN (SELECT Nart FROM provee);
```

3.3. Cláusula ORDER BY: ordenamiento

ORDER BY ordena el resultado de la consulta por una o más columnas. Sin esta cláusula, el orden devuelto por el motor no está garantizado, aunque suele coincidir con el primer índice utilizado.

```
ORDER BY expresion_orden1 [ASC|DESC]
        [, expresion_orden2 [ASC|DESC]] [, ...];
```

La expresión puede ser un nombre de columna, una expresión cualquiera o el *número de posición* de la columna en el SELECT. ASC es ascendente (por defecto) y DESC descendente. Si se ordena por varias columnas, la segunda actúa como criterio de desempate dentro de la primera, y así sucesivamente.

Ejemplo.

```
-- Listado de articulo - proveedor - precio, ordenado por Nart ascendente.
SELECT DISTINCT Nart AS numero_art,
                pv.Nprov AS numero_prov,
                nombre,
                precio_venta
FROM Proveedores pd, Provee pv
WHERE pd.Nprov = pv.Nprov
ORDER BY Nart;
```

3.4. Funciones de agregación

Las *funciones agregadas* reciben una expresión de columna y devuelven un único valor calculado sobre todas las filas. Las del estándar son:

Función	Devuelve
AVG(expr)	Promedio de los valores no nulos.
COUNT(expr)	Cantidad de valores no nulos. COUNT(*) cuenta filas, incluyendo aquéllas con NULL.
SUM(expr)	Suma de los valores no nulos.
MIN(expr)	Mínimo (compatible con números, fechas y cadenas).
MAX(expr)	Máximo.

Salvo COUNT(*), las funciones agregadas **ignoran los valores NULL**. Esto es importante a la hora de calcular promedios: el denominador es la cantidad de valores presentes, no la cantidad de filas.

Ejemplo simple.

```
SELECT AVG(precio) AS promedio FROM articulos;
```

3.5. GROUP BY: consultas sumarias

GROUP BY convierte un SELECT en una *consulta sumaria*: en lugar de devolver una fila por cada fila de la base, agrupa todas las que comparten el valor de las columnas indicadas y produce una fila por grupo. Las columnas que aparecen en el SELECT y no están dentro de una función de agregación deben aparecer también en el GROUP BY.

Ejemplo.

```
-- Para cada articulo, precio promedio entre todos sus proveedores.
SELECT Nart, AVG(precio_venta) AS promedio_p_v
FROM Provee
GROUP BY Nart;
```

3.6. HAVING: filtros sobre grupos

HAVING es a los grupos lo que WHERE es a las filas: filtra los grupos producidos por GROUP BY usando condiciones que pueden involucrar funciones agregadas. La regla práctica es:

- Si la condición sólo usa *atributos planos* (no agregados), debe ir en WHERE (más eficiente: filtra antes de agrupar).
- Si la condición involucra *funciones agregadas*, debe ir en HAVING (sólo se puede evaluar después de armar los grupos).

Ejemplo.

```
-- Artículos cuyo precio promedio de venta entre proveedores es < 1000.
SELECT Nart, AVG(precio_venta)
  FROM Provee
 GROUP BY Nart
HAVING AVG(precio_venta) < 1000;
```

3.7. Subconsultas

Una *subconsulta* es un SELECT usado dentro de otra sentencia. Pueden aparecer en distintos lugares y con distintas semánticas:

- En el WHERE junto con IN, NOT IN, EXISTS, NOT EXISTS u operadores de comparación (=, <, ANY, ALL).
- En el FROM como *tabla derivada*, dándole un alias para tratarla como una relación más.
- En el SELECT como subconsulta escalar, devolviendo un único valor por fila.

Subconsultas correlacionadas. Son aquellas en las que la subconsulta hace referencia a una columna de la consulta exterior. Se ejecutan conceptualmente una vez por cada fila externa. Aparecen típicamente en patrones del tipo “existe alguien con ...” o “no existe ninguno con ...”.

Ejemplo de NOT EXISTS.

```
-- Clientes que no tienen ninguna factura asociada.
SELECT *
  FROM cliente c
 WHERE NOT EXISTS (SELECT 1
                   FROM factura f
                  WHERE f.nro_cliente = c.nro_cliente)
 ORDER BY c.apellido DESC, c.nombre DESC;
```

Esta consulta es exactamente la del Ejercicio 3.a del práctico.

3.8. Reuniones (JOIN)

Los JOIN se escriben dentro del FROM y se corresponden con las operaciones de reunión del álgebra relacional vista en Bases de Datos I. Su sintaxis general es:

```
tabla1 [NATURAL] { {LEFT|RIGHT|FULL} OUTER | INNER } JOIN tabla2
      [ON condicionJoin] [USING (columna1 [, columna2, ...])]
```

- INNER JOIN: devuelve sólo las filas que matchean en ambas tablas. Es lo que hace, implícitamente, el viejo estilo FROM a, b WHERE a.k = b.k.
- LEFT OUTER JOIN: conserva todas las filas de la tabla izquierda, completando con NULL cuando no hay coincidencia a la derecha. Útil para detectar “filas huérfanas”.
- RIGHT OUTER JOIN: simétrico, conserva todas las de la derecha.

- **FULL OUTER JOIN**: conserva todas las filas de ambas (no soportado en MySQL nativamente, pero sí en PostgreSQL y Oracle).
- **NATURAL JOIN**: une por todas las columnas con el mismo nombre. Cómodo, pero peligroso, porque depende del nombrado.
- **USING (col)**: como el natural pero limitado a las columnas explícitas.

Ejemplo equivalente.

```
-- Estilo "moderno" con INNER JOIN.
SELECT p.Nart, pv.Nprov, p.nombre, pv.precio_venta
FROM proveedores p
INNER JOIN provee pv ON pv.Nprov = p.Nprov;

-- Estilo "tradicional" con coma y WHERE (equivalente).
SELECT p.Nart, pv.Nprov, p.nombre, pv.precio_venta
FROM proveedores p, provee pv
WHERE pv.Nprov = p.Nprov;
```

3.9. Operaciones sobre conjuntos

SQL permite combinar resultados de varios **SELECT** con operadores de conjuntos, siempre y cuando ambas consultas devuelvan la misma cantidad de columnas y de tipos compatibles:

- **UNION**: unión, eliminando duplicados.
- **UNION ALL**: unión sin eliminar duplicados (más eficiente).
- **INTERSECT**: intersección.
- **EXCEPT** (o **MINUS** en Oracle): diferencia.

3.10. INSERT

INSERT agrega filas a una tabla. Tiene dos formas habituales:

```
-- (a) Insertar un registro especificando valores.
INSERT INTO destino (campo1 [, campo2, ...])
VALUES (valor1 [, valor2, ...]);

-- (b) Insertar resultados de una consulta.
INSERT INTO destino (campo1 [, campo2, ...]) seleccion;
```

Si no se nombran las columnas, se asume que se está cargando *toda* la tabla en el orden de definición. Las cadenas se delimitan con comillas simples y las fechas suelen escribirse en formato 'YYYY-MM-DD'.

Ejemplo.

```
INSERT INTO articulos (Nart, descripcion, precio, stock_max, stock_min)
VALUES (5, 'Vino x1L', 2500, 90, 20);
```

3.11. UPDATE

UPDATE cambia valores de filas existentes. Su sintaxis es:

```
UPDATE tabla
SET campo1 = valor1 [, campo2 = valor2, ...]
[WHERE condicion];
```

Cuidado: si se omite el **WHERE**, la modificación se aplica a todas las filas de la tabla.

Ejemplo.

```
-- Aumenta un 20% solo a los articulos baratos.
UPDATE articulos
  SET precio = precio * 1.2
 WHERE precio < 2000;
```

3.12. DELETE

DELETE elimina filas de una tabla:

```
DELETE FROM tabla [WHERE condicion];
```

Sin WHERE elimina *todas* las filas (la tabla queda vacía pero sigue existiendo, a diferencia de DROP TABLE). Si la fila a borrar está referenciada por una clave foránea, el resultado depende de la acción declarada (CASCADE, RESTRICT, SET NULL, SET DEFAULT, NO ACTION); ver Sección 4.3.

4. DDL: definición del esquema**4.1. Tipos de datos**

El estándar SQL clasifica los tipos en 13 categorías primarias. Cada motor puede agregar variantes y sinónimos. Los tipos del estándar SQL-89 son INTEGER, SMALLINT, CHARACTER, DECIMAL, NUMERIC, REAL, FLOAT y DOUBLE PRECISION; SQL-92 agrega CHARACTER VARYING (VARCHAR), DATE, TIME, BIT, TIMESTAMP, INTERVAL y BIT VARYING; SQL:1999 incorpora características objeto-relacionales.

Tipos numéricos (MySQL).

Tipo	Descripción
TINYINT	Entero pequeño −128.,127 (sin signo 0.,255).
SMALLINT	−32768.,32767.
MEDIUMINT	−8,388,608.,8,388,607.
INT	−2,147,483,648.,2,147,483,647.
BIGINT	Hasta $\sim 9,2 \times 10^{18}$.
FLOAT	Coma flotante de precisión simple.
DOUBLE	Coma flotante de precisión doble.
DECIMAL(p,e)	Decimal exacto con p dígitos y e decimales.

Tipos de fecha y cadena.

Tipo	Descripción
DATE	Fecha (YYYY-MM-DD).
DATETIME	Fecha y hora.
TIMESTAMP	Fecha y hora con rango acotado, vinculado a la zona horaria del servidor.
CHAR(n)	Cadena de longitud fija.
VARCHAR(n)	Cadena de longitud variable hasta n .
TEXT/BLOB	Textos o binarios extensos.

Para el práctico. Para precios y montos conviene DECIMAL(p,e) (evita errores de redondeo de FLOAT); para identificadores numéricos, INT; para fechas, DATE; para horarios de entrada/salida (Estacionamiento), TIME.

4.2. CREATE TABLE

CREATE TABLE es el comando central de DDL: define una nueva relación con sus columnas y sus restricciones. Su forma general es:

```
CREATE TABLE tabla (
    campo1 tipo [(tamano)] [NOT NULL] [restriccion_inline],
    campo2 tipo [(tamano)] [NOT NULL] [restriccion_inline],
    ...
    [, ligaduraIntegridad1]
    [, ligaduraIntegridad2]
);
```

Cada campo declara su nombre, tipo y, opcionalmente, si admite NULL y restricciones a nivel de columna. Al final del bloque pueden agregarse restricciones de tabla (claves primarias compuestas, foráneas, CHECKs multi-columna, etc.).

Ejemplo del teórico.

```
CREATE TABLE Proveedores (
    Nprov INTEGER NOT NULL CONSTRAINT pkproveedores PRIMARY KEY,
    Nombre VARCHAR(50) NOT NULL CONSTRAINT uniconombre UNIQUE,
    Direccion VARCHAR(50)
);
```

4.3. Ligaduras de integridad

Las *ligaduras de integridad* (también llamadas restricciones o *constraints*) garantizan que los datos cumplan ciertas reglas. Las que maneja SQL para este práctico son:

- **Claves primarias** (PRIMARY KEY): identifican de manera única a cada fila. Implican NOT NULL y UNIQUE. Una tabla tiene a lo sumo una. Pueden ser simples o *compuestas*.
- **Claves secundarias o candidatas** (UNIQUE): aseguran unicidad sobre una o varias columnas, pero permiten NULL.
- **No nulidad** (NOT NULL): el campo no admite NULL.
- **Restricciones de dominio** (CHECK (condicion)): expresan reglas declarativas sobre los valores que un campo puede tomar (rangos, predicados, etc.).
- **Integridad referencial** (FOREIGN KEY ... REFERENCES ...): obliga a que el valor de una columna exista en otra tabla, o sea NULL.
- **Aserciones** (ASSERTION) y **disparadores** (TRIGGER): mecanismos más generales (no se usan en este práctico, pero se mencionan en el teórico).

La sintaxis general de la cláusula CONSTRAINT es:

```
-- Índices de un solo campo:
CONSTRAINT nombre {PRIMARY KEY | UNIQUE | NOT NULL
                  | REFERENCES tablaexterna [(campoexterno)]}

-- Índices multi-campo:
CONSTRAINT nombre {
    PRIMARY KEY (col1 [, col2, ...])
    | UNIQUE      (col1 [, col2, ...])
```

```

| NOT NULL      (col1 [, col2, ...])
| CHECK (condicion)
| FOREIGN KEY (ref1 [, ref2, ...])
    REFERENCES tablaexterna [(extcol1 [, extcol2, ...])]
    [ON DELETE accion] [ON UPDATE accion]
}

```

Acciones referenciales. Cuando se borra o actualiza una fila a la que apunta una clave foránea, la acción a tomar puede ser:

Acción	Comportamiento
CASCADE	Propaga el borrado/actualización a las filas referenciantes.
RESTRICT	Aborta la operación si existen referencias (lo mismo que NO ACTION en muchos motores).
SET NULL	Pone NULL en la columna referenciante.
SET DEFAULT	Pone el valor por defecto.
NO ACTION	Variante de RESTRICT, normalmente diferida al final de la transacción.

Ejemplos prácticos.

```

-- Artículo con CHECKs sobre stock y precio.
CREATE TABLE articulos (
    Nart INTEGER NOT NULL PRIMARY KEY,
    Descr VARCHAR(100),
    Precio FLOAT,
    Cant INTEGER,
    Stock_min INTEGER,
    Stock_max INTEGER,
    CONSTRAINT control_min_max CHECK (stock_min < stock_max),
    CONSTRAINT precio_positivo CHECK (precio > 0)
);

-- Provee con clave primaria compuesta y dos claves foraneas.
CREATE TABLE provee (
    Nprov INTEGER NOT NULL,
    Nart INTEGER NOT NULL,
    Precio_venta FLOAT,
    CONSTRAINT pkprovee PRIMARY KEY (Nart, Nprov),
    CONSTRAINT fkproveedores FOREIGN KEY (Nprov) REFERENCES proveedores,
    CONSTRAINT fkarticulos FOREIGN KEY (Nart)
        REFERENCES articulos ON DELETE CASCADE
);

```

Cómo elegir la acción referencial. En el inciso 1.a del práctico, borrar un *cliente* debe arrastrar sus facturas ⇒ ON DELETE CASCADE. Pero borrar un *producto* que ya fue facturado debe ser rechazado ⇒ ON DELETE RESTRICT. La consigna fija el comportamiento esperado y eso determina la acción.

4.4. Dominios (CREATE DOMAIN)

Un *dominio* es un tipo definido por el usuario sobre un tipo existente, opcionalmente con valor por defecto, no nulidad y CHECK asociado:

```

CREATE DOMAIN nombreDominio [AS] tipoDatos
    [DEFAULT valorPorDefecto]
    [NOT NULL]

```

```
[CHECK (condicion)];
```

Ejemplo del teórico.

```
CREATE DOMAIN enteroPositivo AS INTEGER
    DEFAULT 2
    CHECK (VALUE > 1)
    NOT NULL;
```

Soporte por motor. CREATE DOMAIN es nativo en **PostgreSQL**, pero **MySQL** y **Oracle** no lo implementan tal cual el estándar. La cátedra lo emula con la combinación *tipo de dato* + *CHECK* (en MySQL también ENUM para conjuntos cerrados como {gris, negro, azul} en el inciso 1.b).

4.5. ALTER TABLE

Permite modificar la estructura de una tabla ya creada:

```
ALTER TABLE tabla {
    ADD {COLUMN campo tipo [(tamano)] [NOT NULL]
        | clausulaConstraint}
    | DROP {COLUMN campo | CONSTRAINT clausulaEliminar}
};
```

Ejemplos.

```
ALTER TABLE Proveedores ADD Telefono VARCHAR(20) NOT NULL;
ALTER TABLE Proveedores DROP Direccion;
ALTER TABLE Proveedores ADD CONSTRAINT telefonounico UNIQUE (Telefono);
ALTER TABLE Proveedores DROP CONSTRAINT telefonounico;
```

4.6. Índices

Un *índice* es una estructura auxiliar (típicamente B-tree o hash) basada en una o más columnas, cuya función es acelerar el acceso a los datos. Los motores lo mantienen automáticamente conforme se insertan, modifican o borran filas.

```
CREATE [UNIQUE] INDEX indice ON tabla
    (campo1 [ASC|DESC] [, campo2 [ASC|DESC], ...])
    [WITH {PRIMARY | DISALLOW NULL | IGNORE NULL}];
```

Ideas principales.

- UNIQUE INDEX: además de acelerar, impide duplicados.
- PRIMARY KEY y UNIQUE crean índices implícitamente.
- Cada INSERT/UPDATE/DELETE cuesta más con cada índice; los índices aceleran consultas pero penalizan modificaciones.

4.7. Vistas (CREATE VIEW)

Una *vista* es una relación virtual, definida como una consulta nominada. No forma parte del modelo lógico, pero se ve y se usa como una tabla más:

```
CREATE VIEW <nombre_vista> AS <expresion_de_consulta>;

-- Ejemplo.
```

```
CREATE VIEW nombre_proveedores AS
SELECT nombre FROM proveedores;
```

Sus usos clásicos son: ocultar columnas sensibles (seguridad), simplificar consultas frecuentes y desacoplar el esquema lógico de las aplicaciones que lo consultan. Su *actualización* (UPDATE sobre una vista) es problemática y depende de cuán mapeable sea la vista a una sola tabla base.

4.8. Borrado de estructuras: DROP

DROP elimina objetos definitivamente. Su sintaxis básica es:

```
DROP {TABLE tabla | INDEX indice ON tabla};
```

A diferencia de DELETE, que vacía una tabla pero la conserva, DROP TABLE la elimina completamente, incluyendo su definición.

5. Conexión con los ejercicios del práctico

Los tres ejercicios del práctico recorren los temas anteriores en el orden natural: primero el esquema (DDL), después los datos (INSERT) y finalmente las consultas (SELECT con todas sus cláusulas).

Ejercicio 1 — Definición de esquemas

Tres bases de datos independientes (practico1a, practico1b, practico1c) que ejercitan distintos rincones de DDL:

- **Inciso a** (MySQL y PostgreSQL). Cliente-Producto-Factura- ItemFactura. Foco: PRIMARY KEY simple y compuesta, CHECK (precio >0, stock_minimo <= stock_maximo) y FOREIGN KEY con acciones distintas (ON DELETE CASCADE para facturas de un cliente borrado, ON DELETE RESTRICT para productos ya facturados).
- **Inciso b** (MySQL y Oracle). Vehículo-Persona-Dueño- Estacionamiento-Parquímetro. Foco: CREATE DOMAIN (nombreYApellido como VARCHAR(45)), restricción de conjunto cerrado (color en {gris, negro, azul} con ENUM en MySQL o CHECK en Oracle), CHECK de rango (altura BETWEEN 0 AND 5000), autoincrementales (AUTO_INCREMENT en MySQL, SEQUENCE + TRIGGER o IDENTITY en Oracle) y ON DELETE RESTRICT para vehículos estacionados.
- **Inciso c** (PostgreSQL y Oracle). Cliente-Automóvil- Categoría-Taller-Accidente. Foco: dominio de marcas restringido ({FIAT, RENAULT, FORD}), rango sobre el modelo (1990–2015), control sobre la tarifa (positiva y menor a 10000) y ON DELETE CASCADE para los accidentes ante el borrado de un cliente.

Ejercicio 2 — Carga de datos coherentes

Para cada una de las tres bases, se insertan datos consistentes que respeten las claves primarias, las foráneas y los CHECK. “Coherente” aquí implica además que los datos alcancen para que las consultas del Ejercicio 3 produzcan resultados no triviales: si pediremos clientes sin facturas, debe existir alguno; si pediremos clientes con más de tres accidentes, también.

Ejercicio 3 — Consultas

Cada inciso ejercita un patrón distinto de SELECT:

- a) “Clientes sin facturas” es un caso típico de *antijoin*: se resuelve con `NOT EXISTS` o `NOT IN` (o `LEFT JOIN ... WHERE ... IS NULL`). Se cierra con un `ORDER BY` apellido DESC, nombre DESC.
- b) “Vehículos que usaron el parquímetro 9” requiere un `JOIN` entre *vehiculo* y *estacionamiento* más un filtro sobre `nro_parquimetro = 9`. Se devuelven sólo *modelo* y *color*, casi siempre con `DISTINCT` para evitar repeticiones por cada estacionamiento.
- c) “Clientes con más de 3 accidentes” es el caso clásico de `GROUP BY + HAVING`: se agrupa por DNI, se cuenta y se filtra con `HAVING COUNT(*) > 3`.
- d) “Máximo y mínimo monto de factura por cliente” usa `MAX`, `MIN` y `GROUP BY` `nro_cliente`, junto con un `JOIN` hacia *cliente* para mostrar el nombre.
- e) Tres consultas en álgebra relacional, donde aparezcan todos los operadores: *selección* (σ), *proyección* (π), *unión* ($\cup$), *intersección* ($\cap$) y *producto cartesiano* ($\times$). Es el puente con la materia Bases de Datos I.

Ejemplo resuelto (3.c).

```
SELECT c.dni, c.nombre, c.apellido
FROM cliente c
JOIN accidente a ON a.dni = c.dni
GROUP BY c.dni, c.nombre, c.apellido
HAVING COUNT(*) > 3;
```

Ejemplo resuelto (3.d).

```
SELECT c.nro_cliente,
       c.nombre,
       MAX(f.monto) AS monto_max,
       MIN(f.monto) AS monto_min
FROM cliente c
JOIN factura f ON f.nro_cliente = c.nro_cliente
GROUP BY c.nro_cliente, c.nombre;
```

6. Diferencias entre motores: lo que no hay que olvidar

Tema	MySQL 8+	PostgreSQL 14+	Oracle 10/11 XE
“Crear la base”	CREATE DATABASE	CREATE DATABASE (cada una corre independiente)	Se crea un <i>usuario/esquema</i> : CREATE USER ... y se trabaja sobre él.
Auto incremento	AUTO_INCREMENT en columna	SERIAL / GENERATED ... AS IDENTITY	SEQUENCE + TRIGGER (10/11) o IDENTITY (12+).
Dominios	No soporta CREATE DOMAIN; emular con CHECK o ENUM	Soporta CREATE DOMAIN estándar	No soporta CREATE DOMAIN; emular con CHECK.
Conjuntos cerrados	ENUM('a','b','c')	CHECK (col IN ('a','b','c')) o CREATE TYPE ... AS ENUM	CHECK (col IN ('a','b','c')).
Booleano	BOOLEAN (sinónimo de TINYINT(1))	BOOLEAN real	No tiene; emular con NUMBER(1) y CHECK.
Texto largo	TEXT, LONGTEXT	TEXT	CLOB.

7. Ideas clave para repasar antes del parcial

1. **SQL se divide en DML, DDL y DCL.** Saber rápidamente a qué grupo pertenece cada sentencia ahorra confusiones.
2. **La sentencia SELECT tiene seis cláusulas** y un *orden lógico* de evaluación distinto al orden de escritura. FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY.
3. **WHERE filtra filas; HAVING filtra grupos.** Si no hay agregadas, no hace falta HAVING.
4. **Las funciones de agregación ignoran NULL**, salvo COUNT(*).
5. **NULL se compara con IS NULL / IS NOT NULL**, nunca con =.
6. **LIKE usa % y _** como comodines para patrones de cadenas.
7. **JOIN es la versión moderna del producto cartesiano + WHERE**, y los OUTER JOIN son la forma natural de encontrar “filas sin pareja”.
8. **Una clave primaria es NOT NULL + UNIQUE.** Las claves candidatas se modelan con UNIQUE; las foráneas con REFERENCES.
9. **Las acciones referenciales** (CASCADE, RESTRICT, SET NULL, SET DEFAULT, NO ACTION) son las que bajan los requisitos del enunciado al SQL.
10. **CREATE DOMAIN es estándar y de PostgreSQL;** en MySQL y Oracle se emula con CHECK o ENUM. Ese punto es el que diferencia los tres incisos del Ejercicio 1.
11. **INSERT, UPDATE, DELETE sin WHERE** actúan sobre toda la tabla. Es uno de los errores más comunes y costosos.
12. **DROP TABLE elimina la tabla completa;** DELETE FROM tabla sólo borra las filas.
13. **Patrones típicos de consulta del práctico:**
 - “A sin B” ⇒ NOT EXISTS o LEFT JOIN ... WHERE ... IS NULL.
 - “Cuántos B por A” ⇒ GROUP BY a con COUNT(*).
 - “A con más de k B” ⇒ GROUP BY a HAVING COUNT(*) > k.
 - “Máx/mín por A” ⇒ GROUP BY a con MAX/MIN.

14. **Cada motor tiene sus modismos.** La diferencia más importante a la hora de codificar es la creación del esquema/base, los autoincrementales y el manejo de dominios y enums.

Resumen final

El Práctico 1 es un repaso integral que reúne casi todos los pilares de SQL: DDL para definir tablas con claves, restricciones de dominio y de integridad referencial; DML para poblarlas y consultarlas; JOINS, agrupamientos y funciones agregadas para consultas no triviales; y una práctica concreta de las diferencias entre motores. Si entran sólidas las cinco ideas de “**SELECT** con sus seis cláusulas”, “orden lógico de evaluación”, “**WHERE** vs **HAVING**”, “acciones referenciales” y “dominios estándar vs emulados”, queda muy buena base para encarar el resto de la materia y, en particular, el Práctico 2 (DCL, permisos y roles), que se construye exactamente sobre estos esquemas.